

Swap-Vm Aqua Fees Diff Audit



July 29, 2026

This document has been prepared solely for the named client and it may not be relied upon by any third parties. The content in this document is provided for informational purposes only and does not constitute professional advice of any kind. Security assessments are limited by factors such as scope, time, and techniques used and therefore this document does not constitute a certification, guarantee, or warranty regarding the proper functioning of any system or the absence of security vulnerabilities.

Copyright © 2026 Zeppelin Group Ltd.

Table of Contents

Table of Contents	2
Summary	3
Scope	4
System Overview	5
Security Model and Trust Assumptions	5
Medium Severity	7
M-01 Protocol Fees Are Silently Lost When the Fee Token Blocks the Maker or Recipient	7
Low Severity	8
L-01 ProtocolFeeSkipped Event Is Not Indexed Despite Being the Only Fee-Loss Signal	8
Notes & Additional Information	8
N-01 Non-Aqua Protocol Fee Does Not Validate the Recipient Address	8
Conclusion	10
Appendix	11
Issue Classification	11

Summary

Type	DeFi	Total Issues	3 (0 resolved)
Timeline	From 2026-07-28 To 2026-07-28	Critical Severity Issues	0 (0 resolved)
Languages	Solidity	High Severity Issues	0 (0 resolved)
		Medium Severity Issues	1 (0 resolved)
		Low Severity Issues	1 (0 resolved)
		Notes & Additional Information	1 (0 resolved)
		Client Reported Issues	0 (0 resolved)

Scope

OpenZeppelin conducted a [diff audit](#) of the [1inch/swap-vm](#) repository, covering the changes between the [v1.0.1](#) and [v1.0.2](#) release tags, at base commit [b6e4f97](#) and target commit [32c687c](#).

In scope were the following files:

```
src
├── instructions
└── Fee.sol
```

System Overview

`SwapVM` is a programmable execution engine in which a maker encodes a trade as a sequence of instructions that the engine runs to price and settle a swap. Fees are one instruction family, implemented in `Fee.sol`. Protocol fees charged on the input amount are sent to a configured recipient, and are provided in wallet-funded variants and Aqua-funded variants, where the latter draw on balances tracked by the Aqua accounting layer.

The protocol-fee instructions collect the fee while the program is executing, before the engine transfers the taker's input token to the maker, so the fee is payable only from inventory the maker already holds. This constraint was the motivation for the reviewed change. A position that holds none of the input token could be quoted but could never trade, because the fee transfer reverted, and a thinly funded position was limited in the size it could trade in a single swap. Such one-sided positions became a first-class creation flow in the Aqua application, where a concentrated position can be opened holding only one of a pair's two tokens with its opening price set at a range boundary, so that it begins to trade only once the market price moves into its range.

The change to `Fee.sol`, applied between the `v1.0.1` and `v1.0.2` tags, makes collection of the Aqua protocol fee best-effort so that these positions can trade. The two Aqua-funded variants attempt the transfer inside a `try/catch`, and when it does not succeed the swap proceeds without the fee while a `ProtocolFeeSkipped` event records the omission. Settlement accounting is arranged so that the taker still owes the full input amount in either outcome. The wallet-funded variants are unchanged and continue to revert when the fee cannot be taken.

Security Model and Trust Assumptions

Maker: authors the strategy program, funds the position, and selects the fee instructions, the fee rate, and the recipient. Because the Aqua protocol fee is drawn from the maker's own balance before the taker's input arrives, a maker that does not hold the fee amount, including

the case of a one-sided position that holds none of the input token, does not pay it while the swap still settles. The maker therefore influences whether the Aqua protocol fee is collected, through their balance, allowance, and choice of token.

Taker: supplies the input token and receives the output. Pricing is not adjusted when a fee is skipped, so the taker pays the same amount whether or not collection succeeds.

Protocol and fee recipient: receive protocol fees only when collection succeeds. Collection of the Aqua protocol fee is not guaranteed on-chain, an uncollected fee remains with the maker, and the emitted `ProtocolFeeSkipped` event is the only record of the omission. Detecting uncollected revenue therefore depends on off-chain monitoring of that event.

Tokens and Aqua layer: the fee token and the Aqua accounting layer are assumed to behave as standard. Non-standard tokens, including fee-on-transfer, rebasing, and tokens that restrict specific addresses, fall outside this assumption and can cause the fee transfer to fail while ordinary transfers still succeed.

Integrators: strategies are expected to be validated before use, including that Aqua fee instructions are paired only with Aqua-settled orders and that the fee recipient is configured correctly.

Medium Severity

M-01 Protocol Fees Are Silently Lost When the Fee Token Blocks the Maker or Recipient

Aqua protocol-fee collection attempts best-effort collection through the `_tryPullFee` function. If the fee pull reverts, the swap still settles and a `ProtocolFeeSkipped` event fires instead. However, the `catch` has no filter and swallows every revert from the fee leg, not just the intended case where the maker cannot cover the fee.

`Aqua.pull` sends the fee with `safeTransferFrom(maker, to, ...)`, so the maker is the sender. A maker blacklisted on the fee token, such as **USDT on Ethereum** which blocks *sending* but still allows *receiving*, keeps trading normally. The taker's `tokenIn` is transferred to the maker and settles, while the fee, which is a transfer from the maker, reverts and is silently skipped on every swap. A recipient-side freeze on a token that also blocks receiving, such as **USDC freezing the fee recipient**, has the same effect. This would only revert the whole swap for a fully non-functional token, such as a globally paused one; an address-level block leaves ordinary transfers working and only causes the fee leg to fail.

No user funds are at risk and the taker pays exactly what was quoted, so the impact is lost protocol revenue. However, this behavior can silently zero out collection for every strategy paying fees in the affected token, up to 100%, until the maker or recipient changes. The trigger can be a third party, such as the token issuer, rather than the fee-paying maker.

Consider documenting this scenario, including the behavior of `_tryPullFee` and `ProtocolFeeSkipped`, so that operators understand that address-level blocks or freezes on the fee token can silently stop protocol-fee collection without affecting swap settlement.

Update: *Acknowledged, not resolved. The team stated:*

Maker can definitely do this, but in fact it will be one sided strategy and no longer AMM

Low Severity

L-01 ProtocolFeeSkipped Event Is Not Indexed Despite Being the Only Fee-Loss Signal

When an Aqua protocol fee cannot be collected, the swap still settles and the only on-chain trace is the `ProtocolFeeSkipped` event. Its NatSpec states that off-chain monitoring **MUST** watch it, since it is the only indication that protocol revenue was not collected (see Issue C).

However, none of its parameters (`orderHash`, `token`, `to`, `amount`) are declared `indexed`. Monitors therefore cannot filter the log stream by strategy, token, or recipient, and must fetch and decode every emission to find the ones they care about. This weakens the single control that the best-effort collection model depends on.

Consider marking `orderHash`, `token`, and `to` as `indexed` so that gaps in fee collection can be filtered and detected reliably.

Update: Acknowledged, not resolved. The team stated:

This is not actual for us, we have our own custom rpc and custom indexer so no issues here

Notes & Additional Information

N-01 Non-Aqua Protocol Fee Does Not Validate the Recipient Address

The Aqua and dynamic protocol-fee instructions reject a zero recipient before transferring, via `require(to != address(0), ...)` (for example in `_aquaProtocolFeeAmountInXD` and both dynamic variants). The non-Aqua `_protocolFeeAmountInXD` does not. It calls `safeTransferFrom(maker, to, feeAmountIn)` with an unvalidated `to`.

For most tokens a transfer to `address(0)` reverts, so the swap fails loudly and no harm results. However, a token that treats `address(0)` as a burn address would silently destroy

the fee instead of paying it, and the inconsistency with the sibling instructions is a footgun for strategy builders.

Consider adding the same `to != address(0)` check to the non-Aqua variant for parity with the Aqua and dynamic paths.

Update: Acknowledged, not resolved. The team stated:

| *This comes under maker responsibility.*

Conclusion

SwapVM executes maker-defined instruction programs to price and settle swaps.

OpenZeppelin reviewed a change to its fee instructions that makes Aqua protocol-fee collection best-effort, allowing one-sided positions that hold none of the input token to trade.

The change is small and self-contained, and the settlement accounting leaves takers unaffected when a fee is not collected. The review identified one medium-severity issue, one low-severity issue, and one note, concerning cases where a protocol fee is silently uncollected and the limited on-chain visibility of that outcome. Addressing them would tighten the protocol's fee-collection guarantees and the ability to monitor them.

OpenZeppelin thanks the 1inch team for their cooperation and responsiveness throughout the review.

Appendix

Issue Classification

OpenZeppelin classifies smart contract vulnerabilities on a 5-level scale:

- Critical
- High
- Medium
- Low
- Note/Information

Critical Severity

This classification is applied when the issue's impact is catastrophic, threatening extensive damage to the client's reputation and/or causing severe financial loss to the client or users. The likelihood of exploitation can be high, warranting a swift response. Critical issues typically involve significant risks such as the permanent loss or locking of a large volume of users' sensitive assets or the failure of core system functionalities without viable mitigations. These issues demand immediate attention due to their potential to compromise system integrity or user trust significantly.

High Severity

These issues are characterized by the potential to substantially impact the client's reputation and/or result in considerable financial losses. The likelihood of exploitation is significant, warranting a swift response. Such issues might include temporary loss or locking of a significant number of users' sensitive assets or disruptions to critical system functionalities, albeit with potential, yet limited, mitigations available. The emphasis is on the significant but not always catastrophic effects on system operation or asset security, necessitating prompt and effective remediation.

Medium Severity

Issues classified as being of medium severity can lead to a noticeable negative impact on the client's reputation and/or moderate financial losses. Such issues, if left unattended, have a moderate likelihood of being exploited or may cause unwanted side effects in the system.

These issues are typically confined to a smaller subset of users' sensitive assets or might involve deviations from the specified system design that, while not directly financial in nature, compromise system integrity or user experience. The focus here is on issues that pose a real but contained risk, warranting timely attention to prevent escalation.

Low Severity

Low-severity issues are those that have a low impact on the client's operations and/or reputation. These issues may represent minor risks or inefficiencies to the client's specific business model. They are identified as areas for improvement that, while not urgent, could enhance the security and quality of the codebase if addressed.

Notes & Additional Information Severity

This category is reserved for issues that, despite having a minimal impact, are still important to resolve. Addressing these issues contributes to the overall security posture and code quality improvement but does not require immediate action. It reflects a commitment to maintaining high standards and continuous improvement, even in areas that do not pose immediate risks.